

Adlink Mock Controller

How to use `AdlinkMockController.exe` to play the machine for a third application that runs on the mock `APS168.dll` : switch sensors, watch outputs and axes, and script how the machine reacts.

Written against AdlinkMockController `c570dec` (26 May 2026) and APS168_Mock `aea20b4` (29 Sep 2026).

CONTENTS

§1	How the link works	§2	Before you start	§3	Connect to the application
§4	The main window	§5	Scripts	§6	What crosses between programs
§7	Editing the state file	§8	Troubleshooting	§9	Known limitations

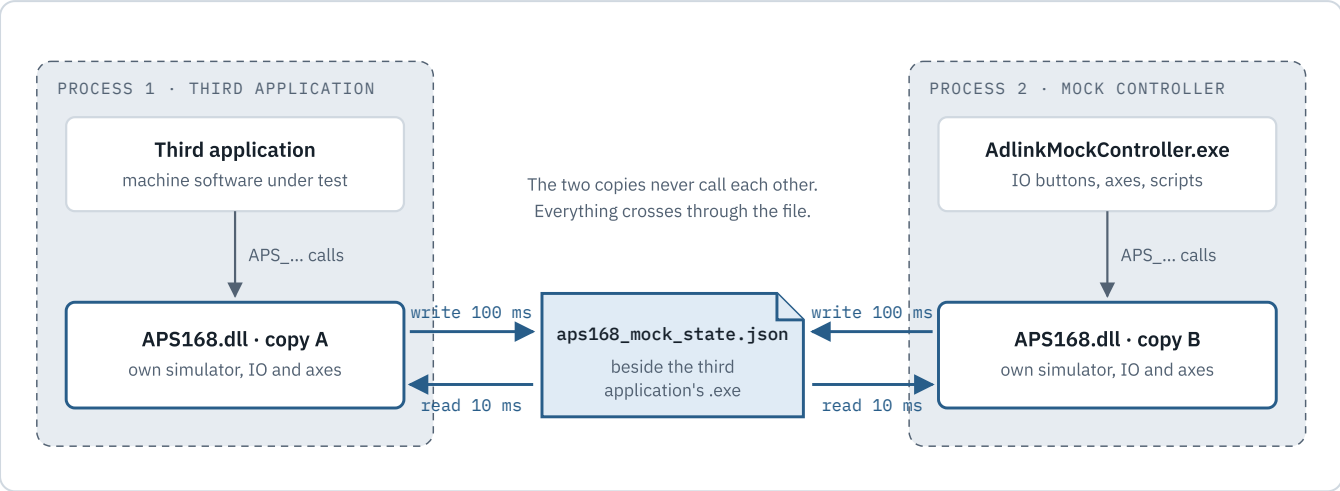
The short version

- 1 Copy the mock DLL into the third application's folder in place of the real ADLINK DLL: `APS168.dll` , or `APS168x64.dll` for a 64-bit application.
- 2 Start the third application and let it initialise the motion card.
- 3 Start `AdlinkMockController.exe` .
- 4 Click `Set Mock State Json File Path` and choose `aps168_mock_state.json` in the third application's folder.
- 5 On **IO Control** › **Input**, click a sensor and check that the application sees it change.

§1 How the link works

The third application (the machine software you are testing) and the controller each load their own copy of the mock `APS168.dll` . The two copies run in different processes and cannot see each other's memory. They share one file instead: `aps168_mock_state.json` , which holds the whole simulated machine (digital IO, axes, servo states, alarms and axis parameters).

- Every **100 ms**, each copy writes its full state to the file. It writes a temporary file and then swaps it in, so a reader never sees half a file.
- Every **10 ms**, each copy reads the file. If the other program changed something, the copy merges that change into its own state.



Both copies of the DLL must use the same file. The controller is pointed at the application's file in §3.

By default each copy uses `aps168_mock_state.json` beside its own program's `.exe`. Out of the box, the controller and the application therefore use two different files and cannot see each other. Pointing the controller at the application's file (§3, step 4) is the one step that connects them.

§2 Before you start

WHERE	WHAT MUST BE THERE	NOTES
Controller folder AdlinkMockController\bin\Debug	AdlinkMockController.exe , AdlinkMockController.exe.config , APS168.dll (mock), APS-4XMO.dll , APS_Define.dll , HSL-DIO.dll , PCI_7856.dll , Newtonsoft.Json.dll	The controller always runs as a 32-bit process, so it needs the 32-bit mock: APS168_Mock\bin\APS168.dll . Do not use APS168x64.dll here.
Controller folder, optional	scripts.json , axes.json	Loaded at startup when present. See §5 and §4.
Third application folder	The mock DLL, under the file name the application loads	32-bit application: APS168.dll . 64-bit application: APS168x64.dll . Both come from APS168_Mock\bin\ . Keep the real ADLINK DLL somewhere safe so you can put it back.

WHERE	WHAT MUST BE THERE	NOTES
C:\Nutek\MachConfig\	MachineConfig.mdb with a DigitalIO table	Required. The controller reads IO names, addresses and NO/NC settings from it at startup and does not start without it.

NOTE

The state file has the same format in both builds, so a 64-bit application and the 32-bit controller can share one file.

The IO database

The controller shows one button per row of the DigitalIO table. It reads the table through Microsoft Jet 4.0, which Windows includes for 32-bit programs.

IO_ID	The point's ID. Scripts refer to IO by this number. Inputs use 0–63, outputs 64 and up.
ModuleNo	EtherCAT module. Inputs are on modules 1 and 3 ; outputs on 5 and 7 .
ChannelNo	Bit within the module, 0–31.
IO_Type	Input or Output .
IO_Name	Label shown beside the button, for example I_SAFETY_RELAY_OFF .
NormallyOpen	False makes the controller invert the raw bit (§4).

§3 Connect to the third application

- 1 Install the mock in the application folder.** Back up the real ADLINK DLL, then copy in the mock build that matches the application (§2).
- 2 Start the third application.** Let it reach the point where it initialises the motion card. The mock creates aps168_mock_state.json beside the application's .exe within 0.1 s of loading the mock, which for most applications happens at their first APS call.
- 3 Start AdlinkMockController.exe .** The path at the top right of the window is the state file the controller is using. At first it is the file in the controller's own folder.
- 4 Click Set Mock State Json File Path** and choose the application's aps168_mock_state.json . The path label changes to that file. The controller immediately takes on everything in it: IO, servo states, positions and parameters.
- 5 Check the link.** On **IO Control > Input**, click a sensor the application displays. The application should see it change within about 0.2 s. Then switch an output from the application and watch its button turn **ON** on the **Output** tab.

CAUTION

Set the path before you click anything else. Until you do, your changes go into the controller's own file and the application never sees them.

NOTE

The controller does not remember the path. Repeat step 4 each time you start it.

Stopping and restarting

- You can close the two programs in any order. Each copy of the DLL saves one last time when its program exits.
- The file keeps the last state of the machine. A program that starts up loads everything in it, including positions, servo states and IO. To start from a clean machine, close both programs and delete `aps168_mock_state.json`.
- If the application calls `APS_close` when it exits, the file records the mock as uninitialised. The controller follows, and motion started from the controller stops moving until the application initialises again.

§4 The main window

The window has three tabs, **IO Control**, **Scripts** and **Motion Control**, plus the state file bar at the top right: the `Set Mock State Json File Path` button and the path of the file in use.

IO Control

Two sub-tabs, **Input** and **Output**. Every point in the `DigitalIO` table gets a button and its name, in database order, 16 to a column and four columns to a tab. Buttons refresh every 100 ms and show `ON` in green or `OFF` in red.

TAB	CLICKING A BUTTON	USE IT TO
Input	Toggles the simulated sensor. The application reads the new value on its next input read.	Fake a sensor: part present, cylinder up, door closed. The application cannot change inputs, so a sensor stays where you put it.
Output	Forces the output to the other state. The application may overwrite it the next time it writes its outputs.	Watch what the application is driving. Forcing is mainly for quick tests.

ON AND OFF ARE LOGICAL STATES

For a point with `NormallyOpen = False`, the controller inverts the raw bit, so `ON` means the raw bit is 0. In the current database these points are `I_ESTOP`, `I_DOOR_OPEN`, `I_SEC_STOPPER` and `O_PCB_STOPPER_SV`. Scripts use the same logical states.

Motion Control

One panel per axis, titled Axis 0: Axis 0 and so on for axes 0–7 unless `axes.json` says otherwise.

CONTROL	WHAT IT DOES
SERVO ON / SERVO OFF	Shows whether the axis servo is on, whoever switched it. Click to switch the servo on. It cannot switch the servo off, because the mock always switches it on. To switch a servo off, set <code>"servoOn": false</code> for the axis in the state file (§7).
Reset Err	Clears the axis error code and the <code>ALM</code> alarm bit. The application sees the alarm clear on its next status read.
Pos	Feedback position in pulses, as returned by <code>APS_get_position</code> .
Cmd	Command position in pulses, as returned by <code>APS_get_command</code> .

Pos and Cmd follow the axis while the application moves it.

To show fewer axes or give them names, put `axes.json` beside `AdlinkMockController.exe`. The panels then read Axis 0: X and so on. Axis IDs run from 0 to 7. If the file is missing or unreadable, all eight axes appear.

```
[
  { "AxisId": 0, "Name": "X" },
  { "AxisId": 1, "Name": "Y" },
  { "AxisId": 2, "Name": "Z" }
]
```

§5 Scripts

Scripts make the controller react like the real machine. A script watches IO and axes, and when its triggers become true it runs a list of actions. A typical script: when the application opens a cylinder valve, switch the matching "up" sensor on and the "down" sensor off.

The Scripts tab

The toolbar has `New Script`, `Edit`, `Delete`, `Save Scripts` and `Load Scripts`. The text beside them says where the list was last loaded from or saved to.

COLUMN	USE
Coloured button	Shows the script name. Click it to run the actions now, whatever the triggers say. This works even when the script is disabled.
Enabled	Tick to let the script run by itself when its triggers become true.
Name, Mode	The script's name and firing mode.

COLUMN	USE
Trigger Groups, Actions	How many of each the script has.

Double-click a row, anywhere except the button and the checkbox, to edit the script.

Editing a script

The editor has **Name**, **Enabled**, **Color** (the button colour) and **Mode**, then the triggers and the actions. Click **OK** to keep your changes in the list. They are written to disk only when you click **Save Scripts**.

Triggers. Click **+ Add Group (OR)** to add a group. In a group, **+ Row** adds a condition, **- Row** removes the selected condition and **Remove Group** deletes the group. All conditions in a group must be true (AND). The script fires when any one group is true (OR). A script with no conditions never fires by itself; run it with its button.

TYPE	FIELDS	TRUE WHEN
IO	IO, Exp. ON	The IO point's logical state matches Exp. ON . Works for inputs and outputs.
AxisPosition	Axis #, Cmp, Position, Source	The axis position compared with Position passes the Cmp test: Equal , NotEqual , Less , LessOrEqual , Greater or GreaterOrEqual . Source picks Feedback (Pos) or Command (Cmd).
AxisMoving	Axis #, Exp. ON	The axis moving state matches Exp. ON . It only sees motion started by the controller itself, not by the application (§9).

Mode.

Rising edge Fires once when the triggers go from false to true. The triggers must go false again before it can fire again. This is the default.

Continuous Fires on every check while the triggers stay true. If the previous run is still going, that check is skipped, so runs never overlap. Useful for scripts driven by axis position.

Triggers are checked every 100 ms. A moving axis can pass a position between two checks, so prefer **GreaterOrEqual** or **LessOrEqual** to **Equal**. **Equal** is reliable for the end point of a move, because the mock stops exactly on the target.

NOTE

A script forgets its previous trigger state when the controller starts or loads scripts, when you tick Enabled, and when you save an edit to it. If its triggers are already true at that moment, it fires once straight away.

Actions. Click **+ Add Row** to add an action (it starts as a 100 ms Delay; change the Type), **- Remove Row** to delete the selected one, and **▲ Up** / **▼ Down** to reorder. Actions run in order, top to bottom, in the background. Cells that show – don't apply to the chosen type.

TYPE	USES	WHAT IT DOES
IOWrite	IO, ON?	Sets an IO point to logical ON or OFF. On an input it sets the sensor the application reads. On an output it forces the output.
ServoOn	Axis #	Switches the servo on. The mock ignores ON? : this action cannot switch a servo off.
MotionAbsolute	Axis #, Pos, Speed	Moves to position Pos at Speed .
MotionRelative	Axis #, Dist, Speed	Moves by Dist pulses from where the axis is.
MotionVelocity	Axis #, Speed	Moves continuously. A negative speed moves in reverse. Runs until a stop action or a soft limit.
MotionHome	Axis #	Moves to position 0 at the axis's last move speed.
MotionStop	Axis #	Stops the axis.
MotionEmgStop	Axis #	Emergency-stops the axis and sets the CSTP status bit.
ResetError	Axis #	Same as Reset Err .
InjectMotionStatus	Axis #, Mask, ON?	Sets (ON? ticked) or clears one motion-status bit. Pick the bit from the Mask list.
InjectMotionIOStatus	Axis #, Mask, ON?	Sets or clears one motion-IO bit, such as ALM , PEL , MEL , ORG or EMG .
Delay	Delay (ms)	Waits before the next action.

Motion actions need the axis servo on and the mock initialised (the application has called **APS_initial**). Otherwise the axis doesn't move.

- Speeds are in pulses per second. The mock moves at constant speed, with no acceleration ramp.
- A speed of 0 reuses the axis's last move speed, which starts at 1000 pulses/s.
- When a move ends, the axis settles for 0.5 s before it reports done.
- Soft limits default to $\pm 600\,000$ pulses. A move past a limit stops at the limit.

Injected status bits are written straight into the state file, so the application sees them too. Some bits belong to the mock and can't be injected:

ACTION	BITS THAT WORK	BITS THE MOCK IGNORES
InjectMotionIOStatus	ALM PEL MEL ORG EMG EZ WARN ZSP SPEL SMEL TLC ABSL STA PSD MSD	INP SVON RDY

ACTION	BITS THAT WORK	BITS THE MOCK IGNORES
InjectMotionStatus	Stop reasons and other flags: SVONS EMGS ALMS WANS PELS MELS ECES SPELS SMELS STPOA GDCES LIP CIP VS PMV PDW PPS SLV GTM PAPB	Motion-phase bits: CSTEP VM ACC DEC DIR NSTP HNV SMV JOG ASTP

An injected bit stays set until something clears it: another inject action with ON? unticked,

(for), or an edit to the file.

Saving and loading

- At startup the controller loads `scripts.json` from its own folder.
- always writes that same `scripts.json` , even if you loaded the list from another file.
- replaces the whole list with a file you choose.
- Changes survive only after . The controller doesn't ask before it closes.

Worked example: left shutter

The shipped `scripts.json` includes a pair of scripts that simulate the left shutter cylinder. **Left SD Up** looks like this:

PART	SETTING
Mode	Rising edge
Group 1 (AND)	75: <code>0_LT_SHUT_UP_SV</code> Exp. ON ticked 77: <code>0_LT_SHUT_DN_SV</code> Exp. ON unticked
Actions	1. IOWrite <code>11: I_LT_SHUT_UP_RS</code> ON? ticked 2. IOWrite <code>13: I_LT_SHUT_DN_RS</code> ON? unticked

When the application opens the "up" valve and closes the "down" valve, the controller reports the shutter up. **Left SD Down** is the mirror image. To imitate travel time, clear the "down" sensor first, add a Delay of about 300 ms, then set the "up" sensor.

§6 What crosses between the programs

The merge rules decide what each side can change. Use this table to predict what the application will see.

ITEM	APPLICATION → CONTROLLER	CONTROLLER → APPLICATION	NOTES
Digital inputs	No	Yes	Input tab, IOWrite on an input.

ITEM	APPLICATION → CONTROLLER	CONTROLLER → APPLICATION	NOTES
Digital outputs	Yes	Yes	Merged bit by bit: only the bits that changed are copied, so both sides can change different bits at the same time.
Servo on	Yes	On only	Switching off is only possible by editing the file.
Position and command	Yes	When idle	Each copy accepts a position from the file only while its own axis is idle: not moving and not in the 0.5 s settle after a move. The controller's axes are idle unless a script moved them.
Alarm and limit bits	Yes	Yes	ALM , PEL , MEL , ORG , EMG , WARN and the other bits in §5.
Error code	Yes	Yes	The last writer wins.
Soft limits, axis parameters	Yes	Yes	The last writer wins.
Initialised flag	Yes	No	Set by the application's APS_initial , cleared by APS_close . The controller never changes it.
Moving, done, homing, in-position	No	No	Each copy works these out from the motion it started itself.

What this means in practice

- **The application moves an axis:** the controller shows Pos and Cmd changing, and AxisPosition triggers work. AxisMoving triggers do not see the move.
- **The controller moves an axis** with a script: the application sees the position change, but its own status still reports the axis stopped and in position. Use this for axes the application doesn't drive, or to place an axis before a test. Don't use it to fake a move the application is waiting for.
- **Don't command the same axis from both sides at once.** Each copy owns the position of the moves it started, so the position jumps between the two.

Timing

- A change reaches the other program within about 0.1 s: up to 100 ms until the next write, plus up to 10 ms until the next read.
- The controller's buttons and script checks run every 100 ms. From an application output, through a script, to a sensor the application can see usually takes 0.1–0.3 s, plus any Delay actions.

- A change that is undone within 100 ms may never reach the other side. Keep sensor pulses at least 200 ms long.

§7 Editing the state file by hand

You can edit `aps168_mock_state.json` in any text editor while both programs run. Save the file and both copies of the DLL pick up the change within about 10 ms. The mocks rewrite the file every 100 ms, so reload it in your editor right before each edit and save promptly.

```
{
  "initialized":true,
  "doPorts":[ [0,0,0,0], ... 16 rows ... ],
  "diPorts":[ [0,0,0,0], ... 16 rows ... ],
  "axes":[
    {"id":0,"position":0,"commandPos":0,"accPos":0,"accCmdPos":0,"servoOn":false,
      "motionStatus":32,"motionIOStatus":8,"errorCode":0,
      "softLimitPos":600000,"softLimitNeg":-600000,"softLimitEnable":true,
      "paramsI":{"},"paramsF":{"}}},
    ...axes 1 to 7...
  ]
}
```

Finding an IO bit

`diPorts` holds inputs and `doPorts` holds outputs. Each is a grid of 16 rows by 4 columns of raw bits (1 or 0). For a point with module *M* and channel *C*:

- **Row** = *C* mod 16
- **Column** = 0 for module 1 or 5, 2 for module 3 or 7, plus 1 when *C* is 16 or more

The file holds raw bits, so for a normally closed point, logical ON is 0.

MODULE	GRID	CHANNELS 0–15	CHANNELS 16–31
1 · inputs	<code>diPorts</code>	column 0, row = channel	column 1, row = channel – 16
3 · inputs	<code>diPorts</code>	column 2, row = channel	column 3, row = channel – 16
5 · outputs	<code>doPorts</code>	column 0, row = channel	column 1, row = channel – 16
7 · outputs	<code>doPorts</code>	column 2, row = channel	column 3, row = channel – 16

For example, `I_SAFETY_RELAY_OFF` (module 1, channel 6) is `diPorts` row 6, column 0, and `XYStopperUp` (module 3, channel 31) is `diPorts` row 15, column 3.

Axis fields

<code>servoOn</code>	<code>true</code> or <code>false</code> . Setting <code>false</code> stops any motion immediately. This is the only way to switch a servo off.
<code>position</code> , <code>commandPos</code> , <code>accPos</code> , <code>accCmdPos</code>	Set all four to the same value to move the axis there instantly. Applied only while the axis is idle in both programs.
<code>motionIOStatus</code> , <code>motionStatus</code>	Status bitmasks as decimal numbers. The bits the mock owns (\$5) are ignored.
<code>errorCode</code>	The value <code>APS_get_error_code</code> returns.
<code>softLimitPos</code> , <code>softLimitNeg</code> , <code>softLimitEnable</code>	Soft limits in pulses, and whether they apply.
<code>paramsI</code> , <code>paramsF</code>	Axis parameters by decimal number, for example <code>"36":20000</code> for <code>PRA_VM</code> (0x24).
<code>initialized</code>	Top-level flag. <code>true</code> once the application has called <code>APS_initial</code> . Motion is simulated only while it is <code>true</code> .

scripts.json codes

`scripts.json` stores choices as numbers. You need these only when editing the file by hand.

FIELD	VALUES
FireMode	0 Rising edge · 1 Continuous
Condition Type	0 IO · 1 AxisPosition · 2 AxisMoving
Comparator	0 Equal · 1 NotEqual · 2 Less · 3 LessOrEqual · 4 Greater · 5 GreaterOrEqual
Source	0 Feedback · 1 Command
Action Type	0 IOWrite · 1 ServoOn · 2 MotionAbsolute · 3 MotionRelative · 4 MotionVelocity · 5 MotionHome · 6 MotionStop · 7 MotionEmgStop · 8 ResetError · 9 InjectMotionStatus · 10 InjectMotionIOStatus · 11 Delay

MOVING THE FILE

Any program using the mock can move its state file at runtime with the mock-only function `APS_mock_set_state_path`. The `Set Mock State Json File Path` button calls exactly this. If the application uses it, point the controller at the file the application chose.

§8 Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
The controller stops with an error at startup.	The IO database is missing or has no <code>DigitalIO</code> table, or <code>APS168.dll</code> beside the exe is missing or 64-bit.	Check <code>C:\Nutek\MachConfig\MachineConfig.mdb</code> . Copy the 32-bit mock <code>APS168.dll</code> next to the exe.
Nothing you click reaches the application.	The two programs use different state files.	Compare the path label with the application's folder and repeat §3 step 4.
No <code>aps168_mock_state.json</code> in the application folder.	The application hasn't made an APS call yet, it still loads the real ADLINK DLL, or it moved its file.	Let it initialise the card. Check that the mock DLL is in its folder. Look beside the application's <code>.exe</code> .
A motion action does nothing.	The servo is off, or the mock isn't initialised.	Switch the servo on first (button or ServoOn action). Let the application call <code>APS_initial</code> , or set <code>"initialized":true</code> in the file.
The Servo button won't switch the servo off.	The mock always switches the servo on.	Set <code>"servoOn":false</code> in the file.
A script fires as soon as the controller starts.	Its triggers are already true, and trigger state resets at startup.	Expected. Add a condition, or keep it disabled until you need it.
A position trigger using <code>Equal</code> never fires.	The axis passed the position between two 100 ms checks.	Use <code>GreaterOrEqual</code> or <code>LessOrEqual</code> .
AxisMoving stays false while the application moves the axis.	Moving state doesn't cross between programs.	Use AxisPosition conditions instead.
The application misses a short sensor pulse.	The pulse ended before the next 100 ms write.	Keep pulses at least 200 ms long.
A forced output flips back.	The application wrote its outputs again.	Expected. Change the output from the application instead.
A change occasionally doesn't stick.	Both programs saved at almost the same moment.	Toggle it again.
Scripts are gone after a restart.	They weren't saved, or were loaded from another file.	Click <code>Save Scripts</code> . It writes <code>scripts.json</code> beside the exe.

§9 Known limitations

- **AxisMoving** triggers only see motion started by the controller. Each copy of the mock keeps its own moving and done flags, and those flags don't cross the file.
- **Motion started by the controller** reaches the application as position changes only. The application's status still says the axis is stopped.
- **Servo off** is not available from the controller's button or from ServoOn actions. Edit the file.
- **65th IO point of a type:** each tab shows 64 buttons. The database has 65 outputs, and the last one, `128: O_CENTER_SUPPORT_BR`, is drawn under `O_RESET_SAFETY_RELAY` and cannot be switched from the controller. It still works as a trigger condition.
- **The state file path** is not remembered between sessions.
- **Save Scripts** always writes `scripts.json` beside the exe.
- **Simultaneous saves:** if both programs save within the same instant, the change made just before may be undone.

Adlink Mock Controller manual · describes AdlinkMockController `c570dec` with APS168_Mock `aea20b4`.